



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 08: Modelo-Vista-Controlador

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

28 de Agosto, 2025

Modelo-Vista-Controlador

Motivación: En cada programa podemos encontrar tres responsabilidades principales.

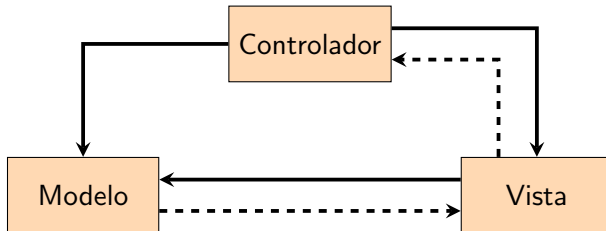
- La **lógica** de la aplicación (qué hacer).
- Los **datos** sobre los que se opera (y cómo se organizan).
- La **presentación** de la aplicación (como se expone al mundo).

Modelo-Vista-Controlador

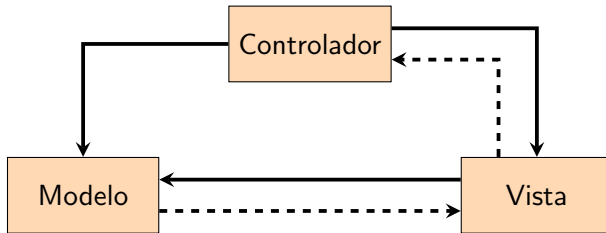
Motivación: En cada programa podemos encontrar tres responsabilidades principales.

- La **lógica** de la aplicación (qué hacer).
- Los **datos** sobre los que se opera (y cómo se organizan).
- La **presentación** de la aplicación (como se expone al mundo).

El patrón MVC separa estas 3 responsabilidades y nos dice cómo debiesen interactuar.



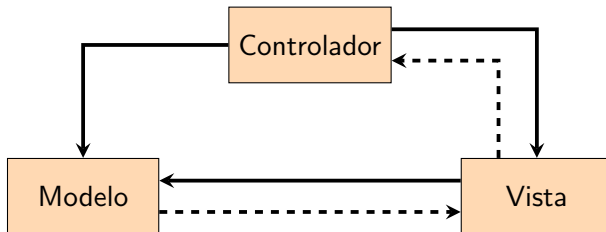
Modelo-Vista-Controlador



Controlador:

- El controlador dirige la orquesta de la aplicación.
- Define qué se hace y cuándo.
- Conoce tanto al modelo como a la vista (o a una interfaz que represente la vista).
- Puede recibir notificaciones de la vista (e.j., se hizo un click).

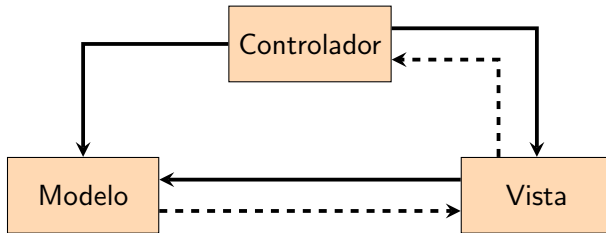
Modelo-Vista-Controlador



Modelo:

- El modelo representa los datos sobre los que se va a operar.
 - Encapsular cualquier forma de almacenamiento de datos.
 - Entregar valor agregado (e.j., hace cálculos sobre los datos).
- El modelo es actualizado/modificado por el controlador.
- El modelo puede notificar a la vista si sus datos cambian.
- El modelo no conoce ni al controlador ni a la vista (evitando acoplamiento).

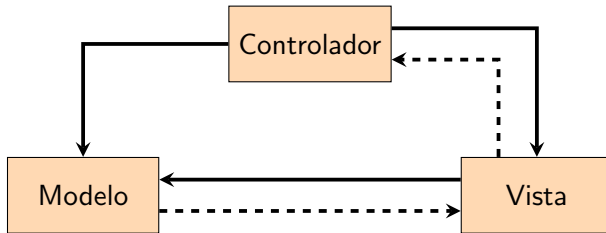
Modelo-Vista-Controlador



Vista:

- La vista es la frontera de comunicación con el mundo exterior.
- La vista está expuesta al usuario.
 - El usuario interactúa con la vista para lograr sus metas.
 - La vista notifica al controlador.
 - El controlador toma las acciones que corresponda.
 - El modelo, de cambiar, notifica a la vista de manera de que actualice su estado.

Modelo-Vista-Controlador



Ventajas:

- Es un buen punto de partida para organizar nuestro código.
- Es fácil cambiar la vista sin modificar el controlador o el modelo.
- Se pueden tener varias vistas (e.j., vista mobile o web).
- El mismo modelo puede ser usado por varias aplicaciones.



Discutamos un ejemplo

Ejemplo: Notas

```
1  ¡Bienvenido!  
2  
3  ¿Qué quieres hacer?  
4  0- Mostrar notas.  
5  1- Agregar una nueva nota.  
6  2- Guardar notas.  
7  3- Salir.  
8  0  
9  ----- Estas son mis notas actuales  
10  
11 -----  
12  
13 ¿Qué quieres hacer?  
14 0- Mostrar notas.  
15 1- Agregar una nueva nota.  
16 2- Guardar notas.  
17 3- Salir.
```

Ejemplo: Notas

```
18 1
19 Ingresa tu nota:
20 Recuerda resolver la E2.
21
22 ¿Qué quieres hacer?
23 0- Mostrar notas.
24 1- Agregar una nueva nota.
25 2- Guardar notas.
26 3- Salir.
27 1
28 Ingresa tu nota:
29 Prepara los documentos del comité de ética.
30
31 ¿Qué quieres hacer?
32 0- Mostrar notas.
33 1- Agregar una nueva nota.
34 2- Guardar notas.
```

Ejemplo: Notas

```
35 3- Salir.  
36 2  
37 Tus notas fueron guardadas exitosamente!  
38  
39 ¿Qué quieres hacer?  
40 0- Mostrar notas.  
41 1- Agregar una nueva nota.  
42 2- Guardar notas.  
43 3- Salir.  
44 3  
45  
46 ¡Hasta pronto!
```

Ejemplo: Notas

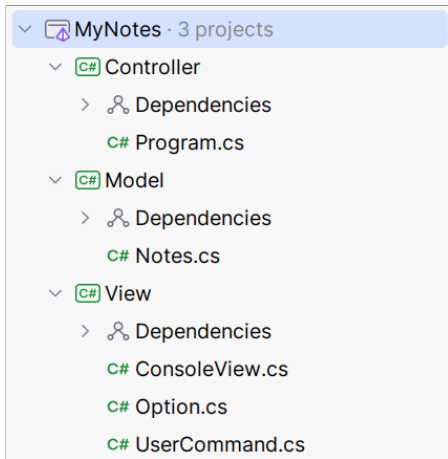
```
35 3- Salir.  
36 2  
37 Tus notas fueron guardadas exitosamente!  
38  
39 ¿Qué quieres hacer?  
40 0- Mostrar notas.  
41 1- Agregar una nueva nota.  
42 2- Guardar notas.  
43 3- Salir.  
44 3  
45  
46 ¡Hasta pronto!
```

Ahora podemos volver a abrir la aplicación y ver que nuestras notas fueron guardadas.

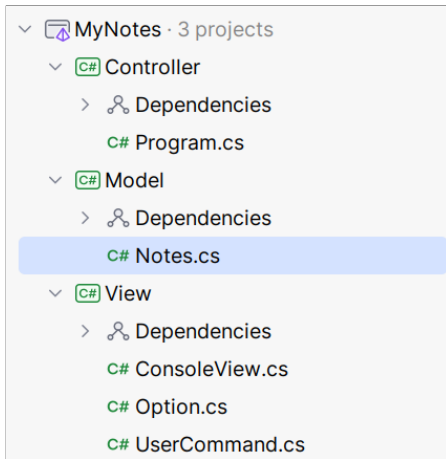
Ejemplo: Notas

```
1 ;Bienvenido!
2
3 ¿Qué quieres hacer?
4 0- Mostrar notas.
5 1- Agregar una nueva nota.
6 2- Guardar notas.
7 3- Salir.
8 0
9 ----- Estas son mis notas actuales
10 Recuerda resolver la E2.
11 Prepara los documentos del comité de ética.
12 -----
13
14 ¿Qué quieres hacer?
15 0- Mostrar notas.
16 1- Agregar una nueva nota.
17 2- Guardar notas.
```

Ejemplo: Notas



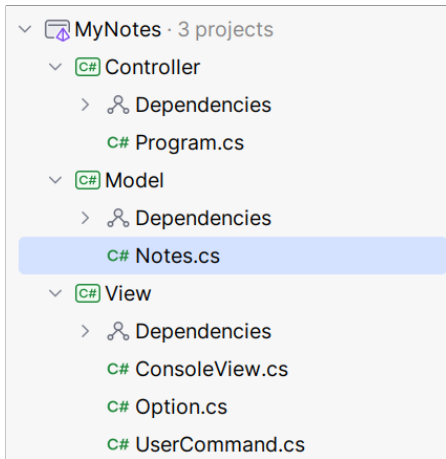
Ejemplo: Notas



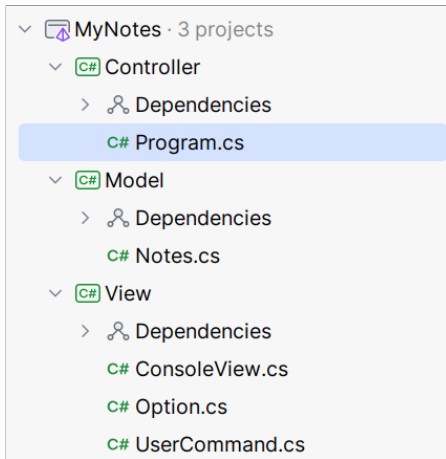
Ejemplo: Notas

```
1 public class Notes{
2     private const string NotesFile = "my_notes.txt";
3     private List<string> _myNotes = new ();
4
5     public Notes() {
6         if(File.Exists(NotesFile))
7             _myNotes = File.ReadAllLines(NotesFile).ToList();
8     }
9
10    public void AddNote(string newNote)
11        => _myNotes.Add(newNote);
12
13    public void SaveNotes()
14        => File.WriteAllLines(NotesFile, _myNotes);
15
16    public override string ToString()
17        => string.Join("\n", _myNotes);
18 }
```

Ejemplo: Notas



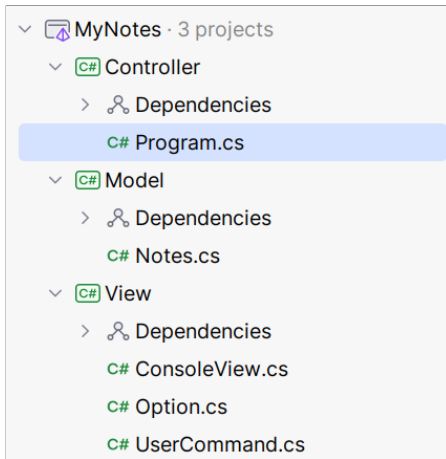
Ejemplo: Notas



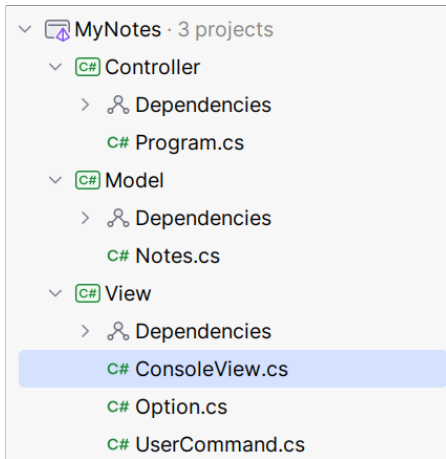
Ejemplo: Notas

```
4 Notes notes = new Notes();
5 ConsoleView view = new ConsoleView(notes);
6
7 view.ShowWelcomeMessage();
8 UserCommand command;
9 do {
10     command = view.AskUserWhatToDoNext();
11     if (command == UserCommand.ShowNotes)
12         view.ShowNotes();
13     if (command == UserCommand.Save) {
14         notes.SaveNotes();
15         view.AnnounceNotesWereSaved();
16     }
17     if (command == UserCommand.AddNote) {
18         string newNote = view.AskUserToInputANote();
19         notes.AddNote(newNote);
20     }
21 } while (command != UserCommand.Exit);
22 view.SayGoodbye();
```

Ejemplo: Notas



Ejemplo: Notas



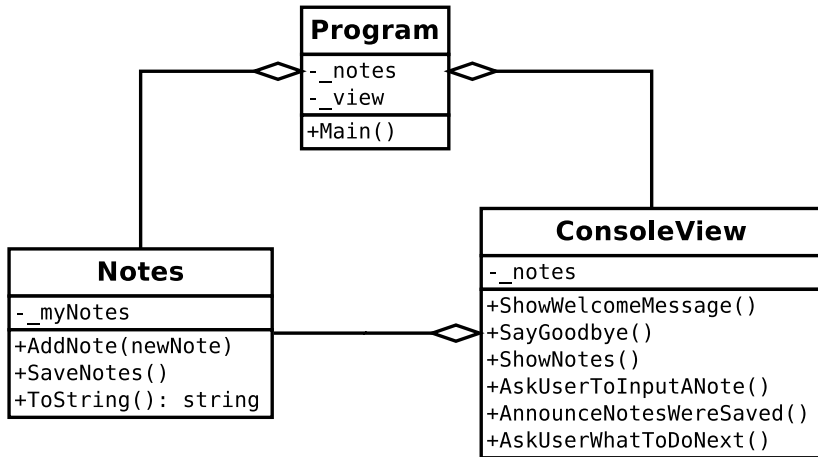
Ejemplo: Notas

```
1 public class ConsoleView {
2     private Notes _notes;
3     private List<Option> _options;
4
5     public ConsoleView(Notes notes) {
6         _notes = notes;
7         LoadOptions();
8     }
9
10    private void LoadOptions() { /* ... */ }
11
12    public void ShowWelcomeMessage()
13        => Console.WriteLine("\n;Bienvenido!");
14
15    public void SayGoodbye()
16        => Console.WriteLine("\n;Hasta pronto!");
17
18    public void ShowNotes() {
19        Console.WriteLine("----- Estas son mis notas actuales");
20        Console.WriteLine(_notes.ToString());
21        Console.WriteLine("-----");
22    }
```

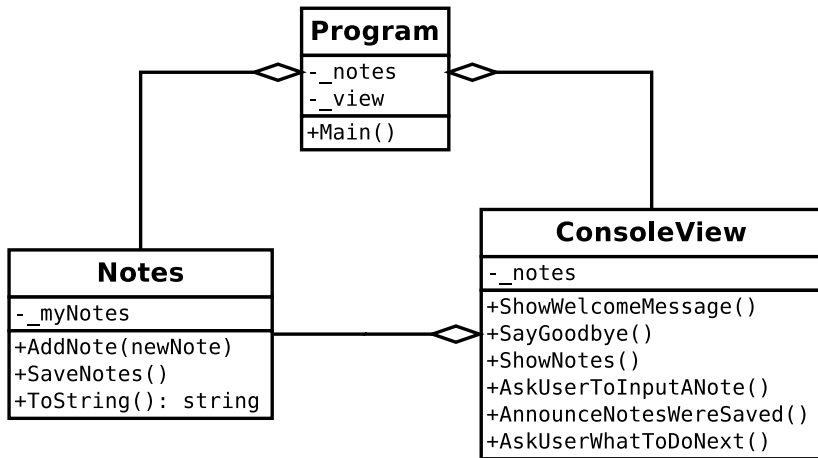
Ejemplo: Notas

```
24 public string AskUserToInputANote() {
25     Console.WriteLine("Ingresa tu nota: ");
26     return Console.ReadLine();
27 }
28
29 public void AnnounceNotesWereSaved()
30     => Console.WriteLine("Tus notas fueron guardadas exitosamente!");
31
32 public UserCommand AskUserWhatToDoNext() {
33     Console.WriteLine("\n¿Qué quieres hacer?");
34     ShowOptions();
35     int optionId = SelectOptionNumber(_options.Count - 1);
36     return _options[optionId].Command;
37 }
38
39 private void ShowOptions() { /* ... */ }
40 private int SelectOptionNumber(int max) { /* ... */ }
41 }
```

Ejemplo: Notas



Ejemplo: Notas



... a simple vista, esta es la idea más tonta que he visto.

Ejemplo: Notas

¿Alguien me puede explicar como es que esto

```
1 Notes notes = new Notes();  
2 ConsoleView view = new ConsoleView(notes);  
3 view.ShowWelcomeMessage();  
4 // ... etc
```

```
1 public class ConsoleView {  
2     /* ... */  
3     public void ShowWelcomeMessage()  
4         => Console.WriteLine("\n;Bienvenido!");  
5 }
```

Ejemplo: Notas

¿Alguien me puede explicar como es que esto

```
1 Notes notes = new Notes();  
2 ConsoleView view = new ConsoleView(notes);  
3 view.ShowWelcomeMessage();  
4 // ... etc
```

```
1 public class ConsoleView {  
2     /* ... */  
3     public void ShowWelcomeMessage()  
4         => Console.WriteLine("\n¡Bienvenido!");  
5 }
```

... es mejor/más claro que simplemente hacer esto??

```
1 Notes notes = new Notes();  
2 Console.WriteLine("\n¡Bienvenido!");  
3 // ... etc
```

Importante: MVC se justifica en proyectos donde la vista cambia mucho o en los que queremos ser capaces de cambiar entre varias vistas.

Importante: MVC se justifica en proyectos donde la vista cambia mucho o en los que queremos ser capaces de cambiar entre varias vistas.

Escenarios posibles:

- 1 Queremos cambiar la vista a inglés.

Importante: MVC se justifica en proyectos donde la vista cambia mucho o en los que queremos ser capaces de cambiar entre varias vistas.

Escenarios posibles:

- 1 Queremos cambiar la vista a inglés.
- 2 Queremos ser capaces de cambiar entre vista inglés y vista español.

Importante: MVC se justifica en proyectos donde la vista cambia mucho o en los que queremos ser capaces de cambiar entre varias vistas.

Escenarios posibles:

- 1 Queremos cambiar la vista a inglés.
- 2 Queremos ser capaces de cambiar entre vista inglés y vista español.

¿Cómo lo hacemos?

Importante: MVC se justifica en proyectos donde la vista cambia mucho o en los que queremos ser capaces de cambiar entre varias vistas.

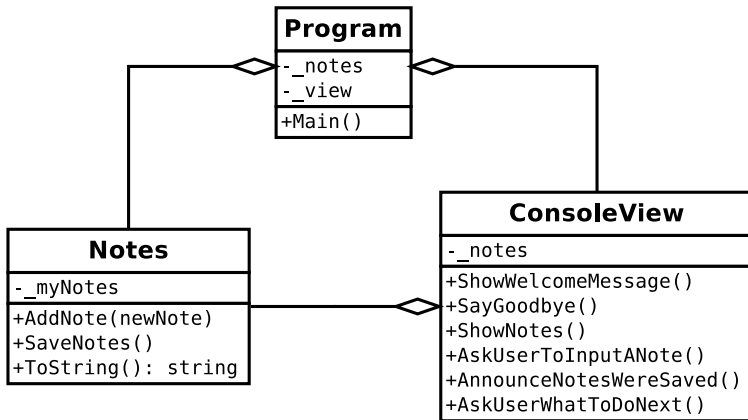
Escenarios posibles:

- 1 Queremos cambiar la vista a inglés.
- 2 Queremos ser capaces de cambiar entre vista inglés y vista español.

¿Cómo lo hacemos?

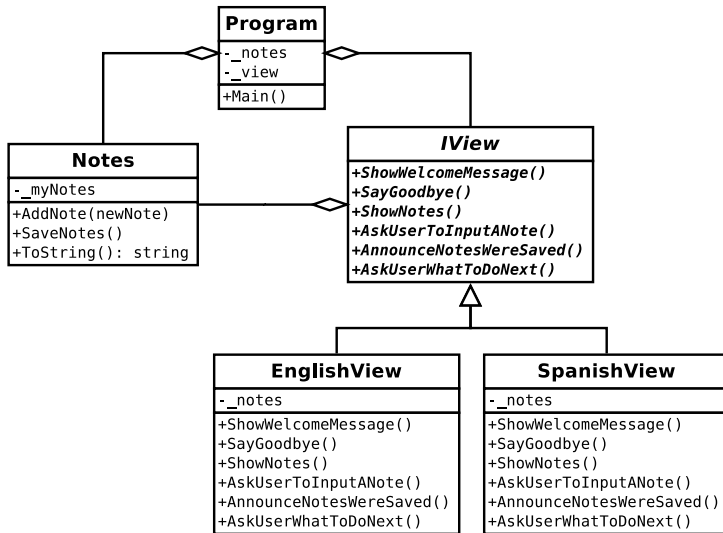
Gracias a que el proyecto está en MVC, no tenemos que cambiar ni el modelo ni el controlador. Basta con agregar *“una nueva vista.”*

Ejemplo: Notas



Noten que el controlador funcionará con cualquier objeto que implemente los mismos métodos públicos que tiene **ConsoleView**.

Ejemplo: Notas



Ejemplo: Notas

```
1 public interface IView {  
2     public void ShowWelcomeMessage();  
3     public void SayGoodbye();  
4     public void ShowNotes();  
5     public string AskUserToInputANote();  
6     public void AnnounceNotesWereSaved();  
7     public UserCommand AskUserWhatToDoNext();  
8 }
```

Ejemplo: Notas

```
1 public interface IView {  
2     public void ShowWelcomeMessage();  
3     public void SayGoodbye();  
4     public void ShowNotes();  
5     public string AskUserToInputANote();  
6     public void AnnounceNotesWereSaved();  
7     public UserCommand AskUserWhatToDoNext();  
8 }
```

```
1 public class SpanishView : IView {  
2     /* ... */  
3     public void ShowWelcomeMessage()  
4         => Console.WriteLine("\n;Bienvenido!");  
5     public void SayGoodbye()  
6         => Console.WriteLine("\n;Hasta pronto!");  
7     /* ... */  
8 }
```

Ejemplo: Notas

```
1 public interface IView {  
2     public void ShowWelcomeMessage();  
3     public void SayGoodbye();  
4     public void ShowNotes();  
5     public string AskUserToInputANote();  
6     public void AnnounceNotesWereSaved();  
7     public UserCommand AskUserWhatToDoNext();  
8 }
```

```
1 public class EnglishView : IView {  
2     /* ... */  
3     public void ShowWelcomeMessage()  
4         => Console.WriteLine("\nWelcome!");  
5     public void SayGoodbye()  
6         => Console.WriteLine("\nBye bye!");  
7     /* ... */  
8 }
```

Ejemplo: Notas

```
1 Notes notes = new Notes();
2
3 IView view;
4 if(Configuration.Language == "English")
5     view = new EnglishView(notes);
6 else if (Configuration.Language == "Spanish")
7     view = new SpanishView(notes);
8 else
9     throw new ApplicationException("Unsupported language exception");
10
11 view.ShowWelcomeMessage();
12 UserCommand command;
13 do
14 {
15     /* ... etc ... */
16 } while (command != UserCommand.Exit);
17
18 view.SayGoodbye();
```

Ejemplo: Domino



Ahora discutamos un ejemplo un poco más complejo.

Ejemplo: Domino

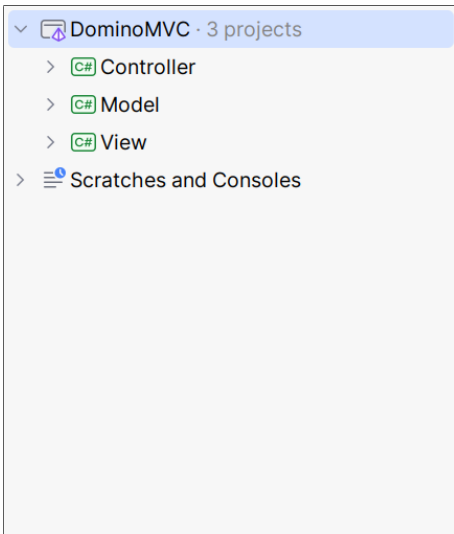
Este proyecto permite jugar dominó en consola:

Ejemplo: Domino

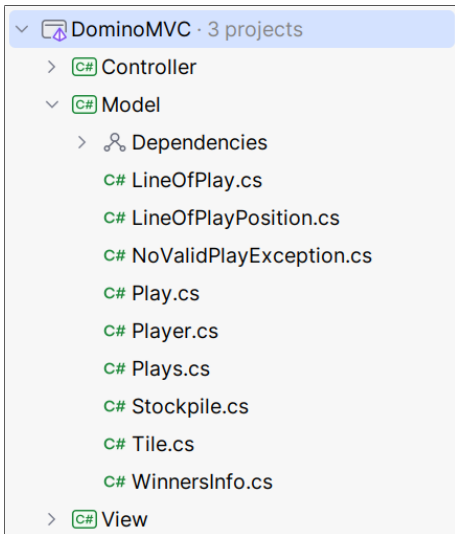
Este proyecto permite jugar dominó en consola:

```
1 Mano jugador 0: [2,4][5,5][1,1][1,3][1,4][3,5][2,5]
2 Jugador 1 tiene 7 fichas en la mano.
3 Jugador 2 tiene 7 fichas en la mano.
4 Jugador 3 tiene 7 fichas en la mano.
5 Jugador 3 tiene 7 fichas en la mano.
6 MESA:
7 Jugador 3 tiene 6 fichas en la mano.
8 Jugador juega [6,6] al inicio de la línea.
9
10 Mano jugador 0: [2,4][5,5][1,1][1,3][1,4][3,5][2,5]
11 MESA: [6,6]
12 No hay fichas que puedas jugar! :(
13
14 Jugador 1 tiene 7 fichas en la mano.
15 MESA: [6,6]
16 Jugador 1 tiene 6 fichas en la mano.
17 Jugador juega [3,6] al inicio de la línea.
```

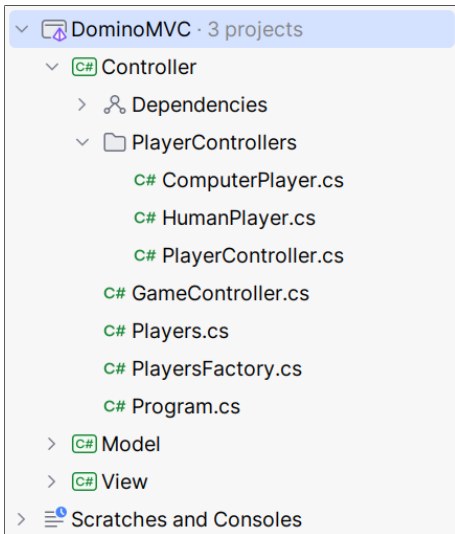
Ejemplo: Domino



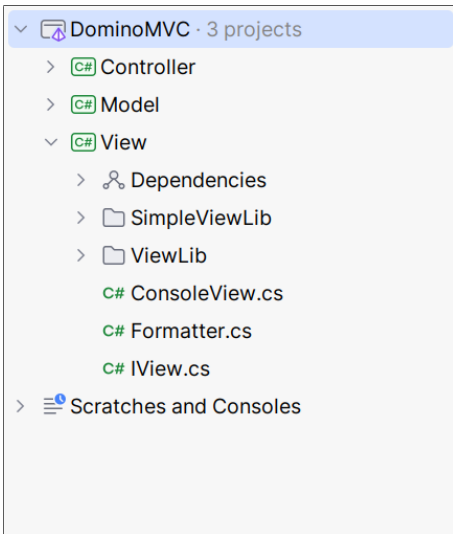
Ejemplo: Domino



Ejemplo: Domino



Ejemplo: Domino



Ejemplo: Domino

▼  DominoMVC · 3 projects

>  Controller

>  Model

▼  View

>  Dependencies

>  SimpleViewLib

>  ViewLib

 ConsoleView.cs

 Formatter.cs

 IView.cs

>  Scratches and Consoles

Ejemplo: Domino

Gracias a MVC, cualquier vista que implemente esta interfaz debería funcionar:

```
1 public interface IView
2 {
3     void Start(Action startProgram);
4     void Show(LineOfPlay lineOfPlay);
5     void ShowHand(Player player);
6     void ShowHandSize(Player player);
7     Play AskUserToSelectPlay(Plays validPlays);
8     void CongratulateWinner(WinnersInfo winners);
9     void AnnouncePlayWasSuccessful(Play play);
10    void AnnouncePlayerMustPass();
11    void AnnouncePlayerPassed();
12 }
```

Ejemplo: Domino

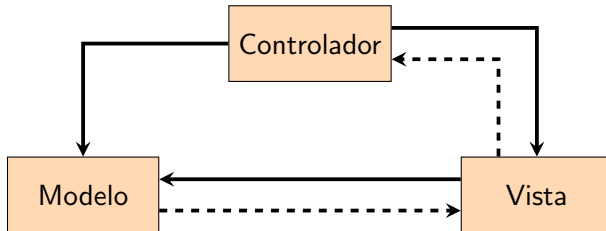
Gracias a MVC, cualquier vista que implemente esta interfaz debería funcionar:

```
1 public interface IView
2 {
3     void Start(Action startProgram);
4     void Show(LineOfPlay lineOfPlay);
5     void ShowHand(Player player);
6     void ShowHandSize(Player player);
7     Play AskUserToSelectPlay(Plays validPlays);
8     void CongratulateWinner(WinnersInfo winners);
9     void AnnouncePlayWasSuccessful(Play play);
10    void AnnouncePlayerMustPass();
11    void AnnouncePlayerPassed();
12 }
```

Obs: Gracias a que la vista conoce al modelo podemos pasar clases completas, por ejemplo Tile, desde el controlador a la vista :)

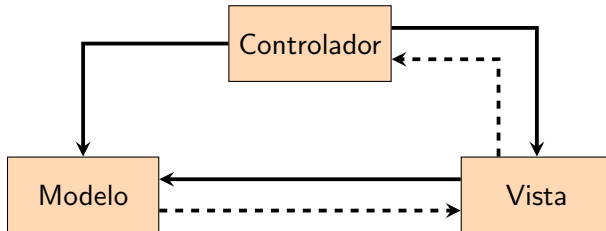
Ejemplo: Domino

Esto me recuerda, ¿cómo decido qué va en cada parte en mi E2?



Ejemplo: Domino

Esto me recuerda, ¿cómo decido qué va en cada parte en mi E2?



Reglas al ojo:

- Toda interacción con el usuario va en la vista (i.e., input/outputs y su formateo).
- Si requiere a la vista para funcionar, es un controlador.
- Si no requiere a la vista, es parte del modelo.

Ejemplo: Domino

Volvamos a esto de la interfaz:

```
1 public interface IView
2 {
3     void Start(Action startProgram);
4     void Show(LineOfPlay lineOfPlay);
5     void ShowHand(Player player);
6     void ShowHandSize(Player player);
7     Play AskUserToSelectPlay(Plays validPlays);
8     void CongratulateWinner(WinnersInfo winners);
9     void AnnouncePlayWasSuccessful(Play play);
10    void AnnouncePlayerMustPass();
11    void AnnouncePlayerPassed();
12 }
```

Ejemplo: Domino

Volvamos a esto de la interfaz:

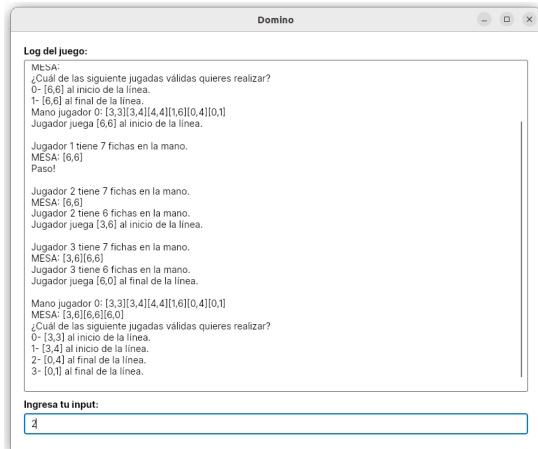
```
1 public interface IView
2 {
3     void Start(Action startProgram);
4     void Show(LineOfPlay lineOfPlay);
5     void ShowHand(Player player);
6     void ShowHandSize(Player player);
7     Play AskUserToSelectPlay(Plays validPlays);
8     void CongratulateWinner(WinnersInfo winners);
9     void AnnouncePlayWasSuccessful(Play play);
10    void AnnouncePlayerMustPass();
11    void AnnouncePlayerPassed();
12 }
```

Obs: Para que el domino funcione con una interfaz gráfica debería basta con implementar IView – sin requerir hacer ningún cambio ni al controlador ni al modelo.

Actividad

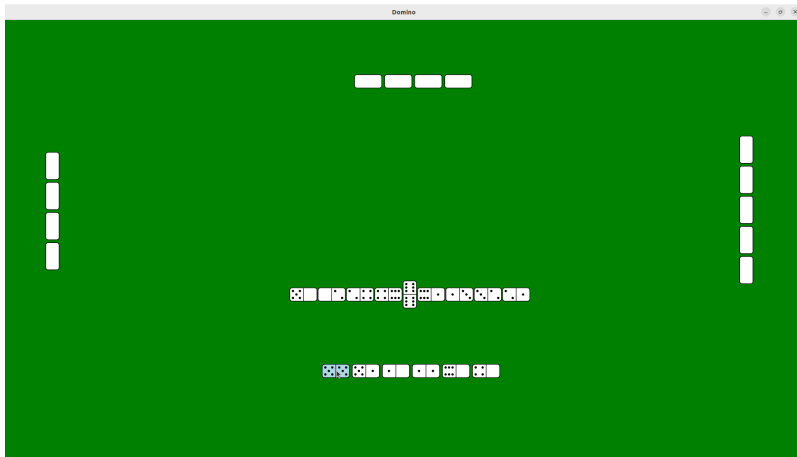
Implementé dos interfaces gráficas para que conecten con el proyecto DominoMVC.

Implementé dos interfaces gráficas para que conecten con el proyecto DominoMVC.

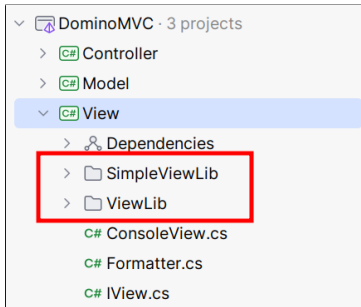


Actividad

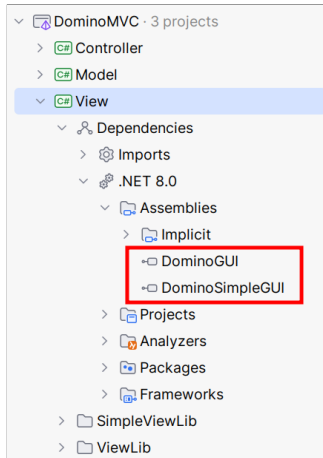
Implementé dos interfaces gráficas para que conecten con el proyecto DominoMVC.



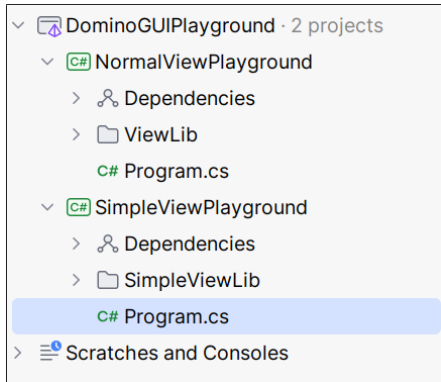
Ambas interfaces ya están incluidas en el proyecto DominoMVC:



Ambas interfaces ya están incluidas en el proyecto DominoMVC:



El proyecto DominoGUIPlayground tiene ejemplos sobre cómo usar las interfaces:



Actividad

Por ejemplo, así se puede usar la primera vista:

```
1 using DominoSimpleGUI;
2
3 DominoWindow dominoWindow = new DominoWindow();
4 dominoWindow.Start(Run);
5
6 void Run() {
7     dominoWindow.Write(";Hola!");
8     dominoWindow.WriteLine(" Este es un ejemplo.");
9     dominoWindow.WriteLine("De lo que se puede lograr con esta vista.");
10    dominoWindow.WriteLine("\nAhora ingresa un input :)");
11    string input = dominoWindow.ReadLine();
12    dominoWindow.WriteLine($"{\nTu input fue: \"{input}\"");
13    dominoWindow.WriteLine("\n¡Adiós!");
14 }
```

Actividad

Por ejemplo, así se puede usar la primera vista:

```
1 using DominoSimpleGUI;
2
3 DominoWindow dominoWindow = new DominoWindow();
4 dominoWindow.Start(Run);
5
6 void Run() {
7     dominoWindow.Write(";Hola!");
8     dominoWindow.WriteLine(" Este es un ejemplo.");
9     dominoWindow.WriteLine("De lo que se puede lograr con esta vista.");
10    dominoWindow.WriteLine("\nAhora ingresa un input :)");
11    string input = dominoWindow.ReadLine();
12    dominoWindow.WriteLine($"{\nTu input fue: \"{input}\"");
13    dominoWindow.WriteLine("\n¡Adiós!");
14 }
```

Notar que se requiere darle el método Run a Start(...).

Actividad

Por ejemplo, así se puede usar la primera vista:

```
1 using DominoSimpleGUI;
2
3 DominoWindow dominoWindow = new DominoWindow();
4 dominoWindow.Start(Run);
5
6 void Run() {
7     dominoWindow.Write(";Hola!");
8     dominoWindow.WriteLine(" Este es un ejemplo.");
9     dominoWindow.WriteLine("De lo que se puede lograr con esta vista.");
10    dominoWindow.WriteLine("\nAhora ingresa un input :)");
11    string input = dominoWindow.ReadLine();
12    dominoWindow.WriteLine($"{\nTu input fue: \"{input}\"");
13    dominoWindow.WriteLine("\n¡Adiós!");
14 }
```

Notar que se requiere darle el método Run a Start(...). Esto es culpa de MacOS.

Actividad

... y nos obliga a hacer eso incluso para la vista en consola (Program.cs):

```
1 using Controller;  
2 using View;  
3  
4 IView view = new ConsoleView();  
5 view.Start(Run);  
6 void Run() {  
7     GameController gameController = new GameController(view);  
8     gameController.Play();  
9 }
```

Actividad

... y nos obliga a hacer eso incluso para la vista en consola (Program.cs):

```
1 using Controller;
2 using View;
3
4 IView view = new ConsoleView();
5 view.Start(Run);
6 void Run() {
7     GameController gameController = new GameController(view);
8     gameController.Play();
9 }
```

El código de la vista en consola llama directo al método dado:

```
1 public class ConsoleView : IView {
2     public void Start(Action startProgram)
3         => startProgram();
4     /* ... */
5 }
```

Actividades:

- **09/09:** Agregar la vista *simple* al DominoMVC.
- **11/09:** Agregar vista *compleja* al DominoMVC

Actividades:

- **09/09:** Agregar la vista *simple* al DominoMVC.
- **11/09:** Agregar vista *compleja* al DominoMVC

Ambas actividades se encuentran disponibles en canvas :)

Actividades:

- **09/09:** Agregar la vista *simple* al DominoMVC.
- **11/09:** Agregar vista *compleja* al DominoMVC

Ambas actividades se encuentran disponibles en canvas :)

HINT: Para la primera actividad, basta con crea una nueva vista que implemente IView. Luego hay que hacer lo mismo que con la vista consola pero llamando a los métodos del write y read del DominoWindow en vez de a la consola.

¿Cuáles son los descuentos típicos en el proyecto debido al MVC?

Descuentos más comunes:

- No se distinguen que clases pertenecen a cada componente.
- El modelo es dependiente de la vista
- El modelo es dependiente del controlador
- El proyecto vista depende del controlador (genera dependencias circulares)
- Se modifican los datos del modelo directamente en la vista
- El controlador depende mucho de que la vista sea por consola (se utiliza `view.WriteLine()` o `view.ReadLine()` directamente en el controlador)